Offline-to-Online Reinforcement Learning with Directed Exploration Models

Anonymous Submission
For Double-Blind Review

***Abstract*—Due to the high cost of exploration in real-world settings, offline reinforcement learning (RL) has become an appealing option for training agents using historical data. However, the restricted coverage of offline datasets may lead to unreliable decisions in unseen states, which calls for the necessity of online fine-tuning to adapt to real environments. Regarding this problem, previous works have focused on ensuring a smooth transition from offline to online RL, but often overlooked how to collect informative samples with minimal interactions. In this work, we propose a novel offline-to-online RL algorithm with Directed Exploration Models (DEM). DEM explicitly leverages a model-based approach to direct the policy to explore regions with high uncertainty and high action values, where high uncertainty regions complement the transition model and high action values facilitate policy optimization. By integrating offline and online data, DEM retains the advantages of fast and cost-effective training of offline RL, while efficiently utilizing online samples. Both theoretical and empirical studies demonstrate the effectiveness of our approach. Experiments conducted on the D4RL benchmark reveal that DEM outperforms most existing offline-to-online RL and model-based offline RL methods.**

***Index Terms*—offline reinforcement learning, directed exploration, model-based learning, online fine-tuning, distribution shift**

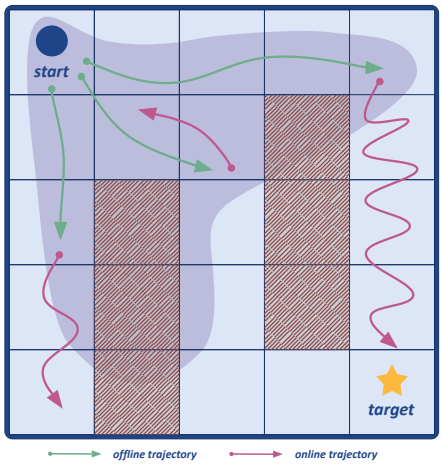

Fig. 1. An illustrative of our motivation. The shaded area represents the transition model's data distribution. Non-shaded areas are considered with high uncertainty. The regions closer to the target state has higher action values.

I. Introduction

Offline reinforcement learning [1] (RL), which can be directly optimized with historical trajectories, has recently achieved promising results in decision-making problems where exploration is costly or even dangerous, such as robot control [2], autonomous driving [3], and healthcare [4]. A primary challenge in offline RL is the distribution shift [5], which arises from the discrepancy between the distribution of the training dataset and that of the online environment. This distribution shift often leads to overestimation of out-of-distribution action values, thereby causing performance degradation. To improve the performance of offline RL policies in specific environments while retaining the fast and cost-effective training benefits of offline RL, offline-to-online RL has recently drawn attention, where the policy is first trained on offline behavior datasets, and subsequently fine-tuned through limited online interactions with the environment, enabling more robust policy adaptation.

When fine-tuning the policy with online interactions, the discrepancy between the offline dataset and online interactions often leads to unstable behaviors of the policy. To bridge the gap between the offline policy and the online environment, current approaches typically enhance the conservatism of RL algorithms during the offline phase, and then relax regularization or pessimistic terms in the online phase to balance exploration and exploitation [6]. Alternatively, prioritized sampling during fine-tuning were adopted to bring the policy closer to the offline data, facilitating a smoother transition between offline and online data [7]–[9].

Although these approaches can alleviate the distribution shift and offer relatively safe exploration, their conservativeness during the offline phase also hinders flexibility during the online phase, preventing the policy from effectively adapting to new data. Moreover, since most offline-to-online RL methods are model-free, they require frequent access to the online environment (typically 200k steps), which incurs high costs and fails to make full use of the available offline data. In contrast, recent advances in model-based RL suggest that the credibility of models can be relatively assured [10]–[14], which motivates us to explore a model-based offline-to-online RL algorithm that can effectively leverage both offline and online data. To fully utilize the online data, we need to complement the transition model with effective samples that require minimal interactions. As presented in Figure 1, when collecting online samples, the right arrow can guide online trajectory to regions with high uncertainty (out of transition model) and high action values (close to the target).

In this work, we propose an offline-to-online RL algorithm with directed exploratory models (DEM). The core idea is

to explicitly guide the policy to explore regions with high uncertainty and high state-action values during the online phase. Exploring high-uncertainty regions helps to improve the generalization of the transition model, while focusing on high action value areas ensures that the collected samples are informative and beneficial for policy optimization. In essence, we adopt an iterative approach: during the offline phase, a transition model is trained to conservatively update the policy; in the online interaction phase, the policy is directed to gather additional samples. Furthermore, we introduce an adaptive weight to emphasize uncertainty during the early learning stages and stability in later stages, thereby smoothly adjusting the policy to meet the dynamic demands of online exploration. Both theoretical and empirical studies demonstrate the effectiveness of our approach. Experiments conducted on the D4RL benchmark [15] reveal that DEM outperforms existing offline-to-online RL and model-based RL methods in most cases.

In summary, the main contributions of our work are as follows:

- We propose an offline-to-online RL method with directed exploratory model, which can collect informative samples for online fine-tuning.
- A dynamically adaptive weight is introduced to smooth the online interaction process, and gradually guide the policy to explore high action value regions.
- Through experiments, we demonstrate that DEM performs well on the widely used D4RL benchmark for RL evaluation with very limited online interactions.

II. Related Works

A. Offline RL

Offline RL learns a policy from offline datasets without interacting with the environment, and can be broadly classified into model-free and model-based methods. The former directly optimizes the policy using offline data, while the latter first learns an environment (transition) model from the offline data and then uses the model to train the policy. The key of the methods is to address the distribution shift problem, which arises from the discrepancy between the offline dataset and the online environment.

In **model-free offline RL**, researchers typically incorporate conservatism into the value function or directly constrain the policy to avoid overestimating the value of out-of-distribution actions [16]–[20]. For example, CQL [21] introduced a regularization term to penalize actions with values exceeding the estimates from the offline data; BCQ [16] combined imitation and Q-learning to learn a policy constrained by the offline data distribution through a generative model. As model-free methods can not generate new data samples, they often suffer from low sample efficiency problem.

Model-based offline RL, on the other hand, begins by constructing a transition model. The policy can then be optimized by interacting with this model [10], [22]–[24]. However, there may be significant extrapolation errors when estimating out-of-distribution data [25], leading to compounded errors. Thus, model-based offline RL approaches focus on exploring out-of-distribution data while maintaining conservatism. For instance, COMBO [12] used conservative policy evaluation, and RAMBO [11] improved conservatism through adversarial training of a pessimistic model. MOPO [10] penalized reward with estimated uncertainty to avoid overestimation. MOREL [26] limited overestimation of state transitions by constructing a pessimistic Markov decision process. Recent studies [14], [27] have also incorporated diffusion model to improve the robustness of the model. While these methods help alleviate extrapolation errors to some extent, they still struggle to adequately evaluate and supplement out-of-distribution data, calling for the necessity of collecting more informative samples.

B. Offline-to-Online RL

Offline-to-online RL often faces the challenge of catastrophic forgetting, where the pre-trained policy struggles to adapt to the online environment and rapidly forgets old knowledge when learning new tasks. Most current methods address this by using more conservative offline RL algorithms during the offline phase and reducing pessimism [6], or smoothing the transition between offline and online data through prioritization or similar techniques during the online fine-tuning phase [7]. AWAC [8] enforced policy imitation of actions with high advantage estimates in the dataset, while PEX [9] used an extended policy to avoid inappropriate actions. More recently, Cal-QL [28] has employed conservative algorithms, such as CQL [21], as a lower bound for the policy's outcomes to prevent performance loss during the online fine-tuning phase. However, these methods are still not sufficiently flexible in balancing conservatism and exploration, and model-free offline settings fail to fully leverage the information inherent in the offline dataset.

III. Preliminaries

A. Markov Decision Process

A Markov decision process (MDP) is defined by the tuple $M = (\mathcal{S}, \mathcal{A}, T, r, \mu_0, \gamma)$. $\mathcal{S}$ and $\mathcal{A}$ denote the state and action spaces respectively, $r(s,a)$ is the reward function, $T(s'|s,a)$ is the transition function, μ_0 is the initial state distribution, and $\gamma \in (0,1)$ is the discount factor.

The state value function $V_M^\pi(s)$, also known as the V-value function, represents the expected discounted return obtained by following policy π starting from state s in the MDP M: $V_M^\pi(s) = \mathbb{E}_{\pi,M}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right]$. The expected discounted return, denoted η_M^π, can be obtained by calculating the expectation of the state value function under the initial state distribution μ_0: $\eta_M^\pi = \mathbb{E}_{s\sim\mu_0}[V_M^\pi(s)]$. The goal in an MDP is to find a policy that maximizes η_M^π.

The state-action value function $Q_M^\pi(s,a)$, or Q-value, represents the expected discounted return when action a is taken in state s, followed by the execution of policy π. We use $\rho_M^\pi(s,a)$ to denote the (improper) discounted state-action visitation distribution under policy π, defined as $\rho_M^\pi(s,a) = \pi(a|s)\sum_{t=0}^{\infty}\gamma^t P(s_t = s|\pi, M)$, where $P(s_t = s|\pi, M)$ is the

probability of reaching state s at time t by following policy π in M. The expected discounted return can be represented as $\eta_M^\pi = \mathbb{E}_{\rho_M^\pi}[r(s,a)]$. We also define the normalized state-action visitation distribution as: $\tilde{\rho}_M^\pi(s,a) = (1-\gamma)\cdot\rho_M^\pi(s,a)$.

B. Model-based Offline RL

In offline RL, the agent can only interact with a static logging dataset $\mathcal{D}_{off} = \{(s_i, a_i, s_i', r_i)\}_{i=1}^N$ collected by the behavior policy π_D. To optimize the policy, model-based offline RL initially constructs a transition model $\hat{T}_\theta(s', r|s,a)$ through supervised learning. This model predicts the next state s' and reward r based on the current state s and action a. The transition model is typically represented as a Gaussian distribution $\hat{T}_\theta(s', r|s,a) = \mathcal{N}(\mu_\theta(s,a), \Sigma_\theta(s,a))$, where $\mu_\theta(s,a)$ represents the mean of the distribution, and $\Sigma_\theta(s,a)$ represents the variance. Subsequently, the policy can be optimized directly by interacting with the supervised transition model. As a result, model-based methods are more sample-efficient, particularly when data collection is challenging. However, the disparity between the real-world and the supervised transition model will accumulate extrapolation error, thereby leading to the distribution shift problem. Therefore, reducing the overestimation of out-of-distribution data is very important in model-based offline RL.

IV. Method

In this section, we present our directed exploration models (DEM) in detail. The core idea is to guide the offline policy to explore regions with high uncertainty and high action values. New samples collected during online interactions are then combined with the offline samples to update the model.

A. Directed Exploration Models

In the offline phase, we first pre-train the policy using the offline dataset $\mathcal{D}_{off}$, along with dynamics models. The dynamics models are trained to predict the next state and reward, while the policy is trained to maximize the expected return.

In the online phase, we interact with the real environment to collect more samples. The collected samples can then be combined to fine-tune the transition model. Due to the high-cost of interactions, we need to ensure that the collected samples are informative and beneficial for policy optimization. By default, as the pre-trained policy is conservative, the samples collected are likely to be concentrated in low-uncertainty regions. To improve the generalization of the transition model, we direct the policy to explore regions with high uncertainty and high action values. Intuitively, high-uncertainty regions complement the transition model, while high action values facilitate policy optimization. Figure 2 illustrates the process of directed exploration.

In specific, we augment the Q-value with the uncertainty estimate of the state-action pair, while maintaining a collection policy π_φ that updates as follows:

$$\begin{aligned}&\nabla_\varphi J_\pi(\varphi)\\&= \mathbb{E}_{(s,a)\sim\mathcal{D}_{on}}\Big[\nabla_\varphi \log \pi_\varphi(a|s)\big(cu(s,a) + Q(s,a)\big)\Big],\end{aligned} \tag{1}$$

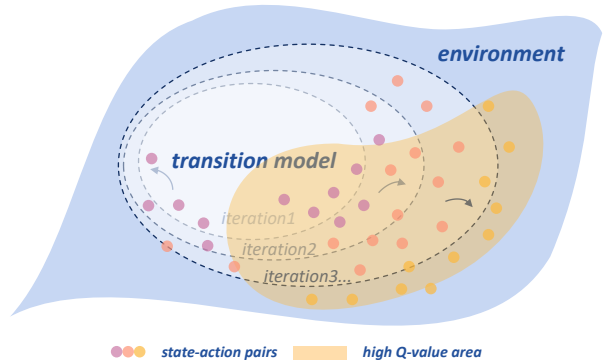

Fig. 2. Directed exploration in the online process. The dashed ellipses indicate the coverage of the transition model trained in different iterations. The area outside the dashed ellipse is the out-of-distribution region with high uncertainty. The orange region has high action value. DEM aims to collect samples that are in the orange region, but out of the ellipses.

where u represents the uncertainty estimate, c is a hyperparameter to adjust the weight of the two parts.

According to the above equation, the policy is updated to maximize the expected return while incorporating the uncertainty estimate of the state-action pair. Let $\mathcal{D}_{on}$ be the set of samples by online interactions. It can be proved the transition model can achieve a higher convergence rate under the uncertainty-compensated reward as long as the transition function is piecewise constant [29]. Intuitively, piecewise constant functions can be thought of as a function that "jumps" one constant value to another at specific points or along certain sets. In many real-world scenarios, functions can exhibit piecewise constant behavior. For example, in some physical systems where there are distinct regions with different constant properties, the corresponding mathematical model might be a piecewise constant function. For the transition model, the agent often encounters different dynamics in different regions of the state-action space, leading to piecewise constant behavior in the model.

Corollary 1. *[29] Suppose the transition function is a piecewise constant function. Then the error reduction rate that the transition function $\hat{T}$ can achieve has a lower bound :*

$$\inf_{\hat{T}} \sup_{T\in PC(\beta,M)} \mathbb{E}_f[\|\hat{T} - T\|^2] \geq c|\mathcal{D}_{on}|^{-\frac{1}{d-1}}, \tag{2}$$

where $c > 0$ is a constant related to the complexity of region $\mathcal{D}_{on}$, d is the intrinsic dimension of $\mathcal{D}_{on}$, and PC(β, M) is the set of piecewise constant functions.

According to the corollary above, the error is bounded by the intrinsic dimension d and the number of samples in $|\mathcal{D}_{on}|$. The error reduction rate is inversely proportional to the intrinsic dimension $d-1$, which is lower than uniform sampling strategy: $c|\mathcal{D}_{on}|^{-\frac{1}{d}}$ [30]. This result suggests that the transition model can be updated more effectively by collecting samples with high uncertainty.

B. Dynamic Weight Adaptation

In our approach, the uncertainty estimation u differs between the offline and online phases. This is because aleatoric uncertainty always exists, whereas epistemic uncertainty can be reduced during online exploration by collecting additional samples.

In the offline phase, the environment dynamics are modeled as a Gaussian distribution: $\hat{T}_\phi(s', r|s, a) = \mathcal{N}\big(\mu_\phi(s, a), \Sigma_\phi(s, a)\big)$, and we use an ensemble of N independently trained models $\{\hat{T}^i_\phi = \mathcal{N}(\mu^i_\phi, \Sigma^i_\phi)\}_{i=1}^N$. We consider the Gaussian predictive variance $\Sigma^i_\phi(s, a)$ as a measure of aleatoric uncertainty. Following MOPO [10], we define the offline uncertainty estimator as:

$$u_{\text{offline}}(s, a) = \max_{i=1,\dots,N} \left\|\Sigma^i_\phi(s, a)\right\|_F, \tag{3}$$

where $\|\cdot\|_F$ denotes the Frobenius norm, capturing the standard deviation predicted by each model on the input (s, a).

In the online phase, the goal is to actively explore high epistemic-uncertainty regions to complement the transition model. Beyond the offline estimator (3), we additionally consider the disagreement among the ensemble's mean predictions. For the predicted means $\mu^i_\phi(s, a)$ from each model, we define the disagreement variance as:

$$\text{DisVar}(s, a) = \frac{1}{d}\sum_{j=1}^{d} \text{Var}_{i=1,\dots,N}\big[\mu^i_{\phi,j}(s, a)\big], \tag{4}$$

where d is the dimension of the predicted state-reward vector, and Var_i denotes the variance over the ensemble. The final online uncertainty estimator is thus given by:

$$u_{\text{online}}(s, a) = \max_{i=1,\dots,N} \left\|\Sigma^i_\phi(s, a)\right\|_F + \sqrt{\text{DisVar}(s, a)}. \tag{5}$$

In Equation 1, $Q(s, a)$ and $u(s, a)$ terms have different magnitudes across domains due to variations in state and reward scales. To facilitate the adjustment of uncertainty weights c for different domains, we perform the following normalization process. In the offline phase, we compute the mean and standard deviation of the $\mathbb{E}_{s,a\sim\mathcal{D}_{off}}[Q(s, a)]$ and $\mathbb{E}_{s,a\sim\mathcal{D}_{off}}[u(s, a)]$ for the state-action pairs in $\mathcal{D}_{off}$. Then, during the online phase, the computed values for the state-action pairs are normalized by subtracting the mean and dividing by the standard deviation. This normalization ensures that the values are on a comparable scale, making it easier to adjust the uncertainty weight c across different domains. Based on the magnitude of the average values, we decrease balancing factor smoothly with policy update steps k as follows:

$$c = c_0 \cdot \frac{1}{1 + e^{(k-k_0)\nu}}, \tag{6}$$

where ν, k_0 and c_0 are hyperparameters that can be selected by the user to control the rate and position of weight change. Specifically, ν controls the rate at which c transitions from its initial value towards the target value; a larger k results in a faster adjustment. k_0 is an offset parameter that determines the midpoint of the weight change, meaning that c reaches the halfway point of its adjustment when $k = k_0$. This strategy provides a higher weight for uncertainty in the early stages of training, and gradually shifts the focus towards expected returns. Focusing on the high action value regions is beneficial for obtaining higher return, and can be theoretically justified. The proof is provided in the AppendixB.

Lemma 1. *Suppose the collected samples have higher action values than the average of the offline dataset. Then the expected return of the updated policy π_{on} is at least as high as the offline policy π_{off} in the real environment, i.e., $\eta_M^{\pi_{on}} \geq \eta_M^{\pi_{off}}$.*

C. Practical Implementation

We outline the practical implementation of DEM in Algorithm 1. As presented, the algorithm consists of offline and online stages. In the offline stage, we train the transition model and offline policy using the MOPO framework (Line 6). In the online stage, we interact with the real environment to collect new samples. By directing the collection policy to explore regions with high uncertainty and high action values, we collect informative samples that can complement the transition model and facilitate policy optimization (Line 11-15). The policy is then updated using the collected samples, and the uncertainty weight is dynamically adjusted to balance exploration and exploitation (Line 16-18). By combining offline and online data, we can effectively update the transition model and policy (Line 20-22), thereby achieving better performance.

In practice, we build an ensemble of K regression models. The uncertainty of the transition model is quantified using the maximum standard deviation across the ensemble $u(s, a) = \max_{i=1}^K \|\Sigma^i_\theta(s, a)\|_F$ [10]. The hyperparameters ν and k_0 can be selected by the user to control the rate and position of weight change. The algorithm is terminated when the policy converges or the maximum number of iterations is reached.

V. Experiments

In our experiments, we designed our experiments to answer the following questions:

- How does DEM compare with existing offline-to-online RL and model-based RL approaches?
- Does the uncertainty-guided modeling approach really focus on the regions we expect?
- Does the dynamic weight adaptation method contribute to the final performance?

A. Experiment setting

a) Data collection.: We evaluate DEM on a large subset of datasets in the D4RL benchmark [15] based on the MuJoCo simulator [31], including three environments (halfcheetah, hopper, and walker2d) and three dataset types (medium, medium-replay, medium-expert), yielding a total of 9 problem settings. Their specific composition is as follows and for all datasets we use the v2 version. **medium**: contains transitions from an early-stopped SAC policy. **medium-replay**: composed of the replay buffer generated during training of the medium

Dataset type	Environment	DEM(ours)	MOPO	COMBO	AWAC	MOREL	CQL	TD3+BC	(NEW)
medium	halfcheetah	$\mathbf{70.5 \pm 4.2}$	42.3	54.2	41.1	42.1	44.4	42.8	78.1
	hopper	65.5 ± 10.9	28.0	97.2	91.0	95.4	86.6	**99.5**	56.5
	walker2d	$\mathbf{86.6 \pm 0.5}$	17.8	81.9	79.1	77.8	74.5	79.7	87.2
medium-replay	halfcheetah	$\mathbf{73.1 \pm 0.4}$	53.1	55.1	-	40.2	46.2	43.3	67.3
	hopper	$\mathbf{96.1 \pm 5.9}$	67.5	89.5	-	93.6	48.6	31.4	95.8
	walker2d	$\mathbf{82.6 \pm 0.3}$	39.0	56.0	-	49.8	32.6	25.2	85
medium-expert	halfcheetah	75.4 ± 13.3	63.3	90.0	41.0	53.3	62.4	**97.9**	63.2
	hopper	102.2 ± 0.8	23.7	111.1	111.9	108.7	111.0	**112.2**	88.1
	walker2d	$\mathbf{111.4 \pm 0.4}$	44.6	103.3	78.3	95.6	98.7	101.1	112

TABLE I
RESULTS FOR THE D4RL BENCHMARK.

Algorithm 1 Offline-to-Online Reinforcement Learning with Directed Exploration Models(DEM)

INPUT: Offline dataset $\mathcal{D}_{off}$, learning rate α;
OUTPUT: Policy π_{on};
Randomly initialize parameters φ, ϕ, θ;
Initialize hyperparameters ν, k_0; Set $k = 0$;
 # offline stage:
Train MOPO with $\mathcal{D}_{off}$, obtain $\{\hat{T}_\phi^i\}_{i=1}^N$ and sample collection policy $\pi_\varphi = \pi_{off}$;
 # online stage:
for $iteration = 0, 1, 2, \ldots$ **do**
  for $epoch = 0, 1, 2, \ldots$ **do**
    Sample state s_0 from the online environment;
    for $t = 0, 1, 2, \ldots$ **do**
      Sample action according to $a_t = \pi_\varphi(s_t)$;
      Execute a_t, get reward r_t and next state s_{t+1};
      Add sample (s_t, a_t, r_t, s_{t+1}) to $\mathcal{D}_{on}$;
    end for
    Update policy parameters according to Equation 1;
    $k = k + 1$;
    Update uncertainty weight c according to Equation 6;
  end for
  Update model $\{\hat{T}_\phi^i\}_{i=1}^N$ with $\mathcal{D}_{off} \cup \mathcal{D}_{on}$;
end for
Randomly draw samples from $\mathcal{D}_{off} \cup \mathcal{D}_{on}$, use SAC to optimize π.

policy. **medium-expert**: includes a mixture of suboptimal and expert data.

b) Experimental implementation.: We primarily follow the SAC [32] and MOPO [10] settings for basic hyperparameters, while the other adjusted hyperparameters mainly include the length of the rollouts h generated by the model and the penalty coefficient λ. We ran DEM on 3 random seeds for each task. In each run, we conducted 50k steps of online interaction, with the weight adjustment rate ν set to 0.001 and the weight adjustment position set to 5000. Considering fairness and consistency, we conducted 1M steps of policy training according to convention. We evaluated the policy every 1k steps. For more details about the hyperparameters settings, please refer to Appendix A.

c) Baselines.: We compared DEM with the offline-to-online RL algorithm AWAC [8], several model-based baseline algorithms including MOPO [10], COMBO [12], and MOREL [26], as well as the classic offline RL algorithms CQL [21] and TD3+BC [33]. Since DEM is built upon MOPO with an added online fine-tuning process, comparing it to MOPO can be seen as an ablation study. All the results were taken from the original papers. The comparison results are presented in Table I. We report the performance during the last 10 iterations of training averaged over 3 seeds. $\pm$ captures the standard deviation over seeds. Each number for the D4RL benchmark using the normalization procedure proposed by [15]. The scores are undiscounted average returns normalized to roughly lie between 0 and 100, where a score of 0 corresponds to a random policy, and 100 corresponds to an expert. We bold the highest mean.

B. Evaluation Results on the D4RL Dataset

As shown in Table I, DEM performs well across all tasks, consistently achieving scores on the *medium-replay* dataset that surpassed those of classic RL algorithms. Even in environments where its performance is less outstanding, it maintains a stable level of results. However, its performance in *medium-expert* environments is less satisfactory. This discrepancy could be due to the nature of the offline data. In the *medium-expert* environment, the data is heavily concentrated in high-reward regions, leading to high expected rewards but low uncertainty. This limits the diversity and quality of the data collected during online interaction, thereby constraining DEM's potential. Fortunately, behavior cloning-based reinforcement learning methods, such as TD3+BC shown in the table, demonstrate excellent performance in high-quality datasets. Incorporating behavior cloning techniques may help address the limitations of DEM in such environments. For example, introducing a behavior cloning loss term during policy updates could enhance DEM's ability to utilize high-reward offline trajectories while preserving its exploration advantages.

Notably, DEM outperforms the MOPO algorithm across all 9 problem settings, highlighting the effectiveness of our online framework. Such results are particularly remarkable given the modest online interaction cost of just 50k steps, significantly lower than the budgets required by traditional offline-to-online RL algorithms, which often exceed 200k steps. This efficiency

makes DEM a practical and cost-effective solution for real-world applications.

C. Model Validation with T-SNE

To examine whether the model, supplemented with online data through the uncertainty exploration process, truly focuses on regions with high expected return and high uncertainty, we conducted the following visualization experiment. Specifically, we visualized three different types of datasets: *hopper-medium*, *hopper-medium-replay*, and *hopper-medium-expert*. First, we sampled 2000 transitions (s, a, s', r) from the offline dataset. Next, we input the offline state s into the model to obtain the model-predicted transitions (s, a_T, s'_T, r_T). Then, we used t-SNE [34] to project $\{s'\}$ and $\{s'_T\}$ onto a two-dimensional plane. Finally, we plotted the scatter points of $\{s'_T\}$, using the Q-value network of the offline policy to estimate the expected return of the predicted action-value pair (s, a_T), and the Q-values were represented as the color of the points. Additionally, we displayed the kernel density estimate of the offline state $\{s'\}$ as a reference. The result is shown in Figure 3.

As shown in the figure, the model predominantly distributes its predictions in low-density edge regions, with the scatter points mostly showing red, indicating high uncertainty and high expected return. This suggests that under the guidance of policy data, the model indeed focuses more on regions with both high uncertainty and high action value. Comparing the three subplots within the figure, the scatter points in the *hopper-medium-replay* environment are more dispersed, while those in the *hopper-medium-expert* environment are more concentrated. This indicates that in the *hopper-medium-replay* environment, online interactions enabled sufficient exploration, supplementing more out-of-distribution data. In contrast, in the *hopper-medium-expert* environment, the high quality and low variance of the expert data made it challenging for online interactions to collect higher action-value data through exploration. As a result, the quality of the supplemented data may not match that of the expert data in the offline dataset, leaving the model insufficiently updated. This further demonstrates that DEM is better suited for environments with evenly distributed data but less applicable to high-quality datasets.

D. Ablation Experiment of Dynamic Weight Adaptation

To validate the advantages of the dynamic weight adaptation algorithm, we conducted an ablation experiment comparing the full DEM with a version where the dynamic weight adaptation is removed (DEM-s).

The hyperparameters for each ablation task were consistent with those used in the D4RL benchmark evaluation experiments, with the only difference being that in DEM-s, the uncertainty weight parameter c is fixed as the balance factor c', meaning that the uncertainty weight no longer decreases as the number of online interaction steps increases but remains constant.

As shown in Table II, the full DEM achieved superior performance on nearly all tasks. Notable improvements were observed in the *hopper-medium* and *halfcheetah-medium-replay* tasks, where DEM achieved scores of 65.5 and 75.4, compared to 53.0 and 36.1 for DEM-s, respectively. However, in the walker2d-medium-expert task, DEM slightly underperformed DEM-s. We hypothesize that this task's highly structured expert data may have led to marginal disruptions in DEM's early exploration phase.

Dataset type	Environment	DEM	DEM-s
medium	halfcheetah	**70.5**	64.7
	hopper	**65.5**	53.0
	walker2d	**86.6**	86.2
medium-replay	halfcheetah	**73.1**	72.6
	hopper	**96.1**	90.2
	walker2d	**82.6**	82.1
medium-expert	halfcheetah	**75.4**	36.1
	hopper	102.2	102.2
	walker2d	111.4	**112.0**

TABLE II
RESULTS OF ABLATION EXPERIMENT.

Clearly, the dynamic weight adaptation plays a crucial role in balancing exploration and exploitation during online interaction. Early in the interaction process, a higher uncertainty weight encourages exploration, leading to better environmental coverage. As interactions accumulate, the weight decreases, enabling the model to focus on high-reward regions and stabilize its performance. This adaptive mechanism allows DEM to outperform DEM-s, which relies on a fixed uncertainty weight that cannot adjust to the dynamic demands of the online interaction process.

VI. CONCLUSION

In this work, we propose a model-based offline-to-online reinforcement learning algorithm that uses uncertainty estimates to maintain conservatism during the offline phase and increase exploration during the online phase, effectively smoothing the mitigation of distributional shift. By combining the model with both offline and online data, our approach fully leverages the information from both phases. Our method has been experimentally validated, demonstrating its effectiveness and offering a novel approach for model utilization in offline-to-online RL. It also provides a new option for uncertainty in exploring out-of-distribution data. Furthermore, we hope that this optimization technique for models can be applied to other model-based research fields, such as image processing, among others.

APPENDIX A
EXPERIMENT DETAILS

A. Model Training

In the offline phase, we followed the previous model training methodology [10], [12], using maximum likelihood to train an ensemble of 7 dynamic models. The best 5 models were selected based on the validation error from 1000 transitions in the training dataset. Each model in the ensemble is a 4-layer feedforward neural network with 200 hidden units per layer. After the final hidden layer, the model uses a two-head

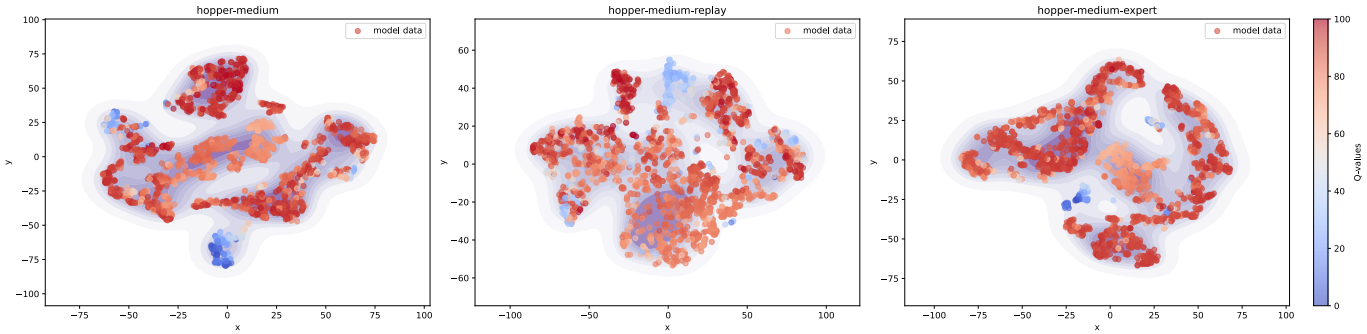

Fig. 3. Comparison between model-predicted states and offline data. We visualized the model-predicted states and offline state data density across three different offline datasets. The scatter points represent the model's data distribution, and the predicted Q-values were normalized and assigned as the color of the scatter points.

architecture to output the mean and variance. In the online phase, the collected data is added to the training dataset, and the model is retrained. For all environments, we performed 50k steps of online interaction and updated the model 5 times.

B. Policy Training

In both the offline and online phases, we used the SAC algorithm for policy updates, following the default parameters of SAC. During the offline phase, we sampled a batch of 256 transitions for policy updates, with 5 % of the data coming from the offline dataset and the rest generated by the model. In the online phase, each update used short trajectory data of 5 rollouts from interactions with the real environment to update the policy 10k times, with a total of 50k interaction steps. After obtaining the new model, the policy is updated in the same manner.

The only parameters we adjusted between datasets are the length of the rollouts h generated by the model, the penalty coefficient λ (mainly referring to MOPO), and the balance factor c_0 selected based on the average Q-value and U-value. For more details, please refer to the table III.

Dataset type	**Environment**	h	λ	c_0
medium	halfcheetah	5	0.5	300
	hopper	5	5.0	300
	walker2d	5	0.5	100
medium-replay	halfcheetah	5	0.5	200
	hopper	5	2.5	300
	walker2d	1	2.5	20
medium-expert	halfcheetah	5	2.5	300
	hopper	5	5.0	300
	walker2d	1	2.5	200

TABLE III
HYPERPARAMETERS USED IN THE D4RL DATASETS.

APPENDIX B
PROOFS

A. Proof of Lemma 1

The proof of Lemma 1 is mainly based on the following telescoping lemma [10]:

Lemma 2. *Let M and $\hat{M}$ be two MDPs with the same reward function r, but different dynamics T and $\hat{T}$ respectively. Let $G^{\pi}_{\hat{M}}(s,a) := \mathbb{E}_{s'\sim\hat{T}(s,a)}[V^{\pi}_{M}(s')] - \mathbb{E}_{s'\sim T(s,a)}[V^{\pi}_{M}(s')]$. Then,*

$$\eta_{\hat{M}}(\pi) - \eta_M(\pi) = \gamma\bar{\mathbb{E}}_{(s,a)\sim\rho^{\pi}_{\hat{T}}}[G^{\pi}_{\hat{M}}(s,a)]. \tag{7}$$

As an immediate corollary, we have

$$\eta_M(\pi) \geq \bar{\mathbb{E}}_{(s,a)\sim\rho^{\pi}_{\hat{T}}}[r(s,a) - |G^{\pi}_{\hat{M}}(s,a)|]. \tag{8}$$

Let π_{off}, $\hat{T}_{off}$ and $\hat{M}_{off}$ be the pretrained policy, transition model and MDP with offline dataset. Let π_{on}, $\hat{T}_{on}$ and $\hat{M}_{off}$ be the updated policy, transition model and MDP after fine-tuning. Our aim is to show that the lower bound improves in the updated policy and transition:

$$\begin{aligned}&\bar{\mathbb{E}}_{(s,a)\sim\rho^{\pi_{on}}_{\hat{T}_{on}}}[r(s,a) - |G^{\pi_{on}}_{\hat{M}_{on}}(s,a)|]\\ &\geq \bar{\mathbb{E}}_{(s,a)\sim\rho^{\pi_{off}}_{\hat{T}_{off}}}[r(s,a) - |G^{\pi_{off}}_{\hat{M}_{off}}(s,a)|].\end{aligned} \tag{9}$$

According to Corollary 1, the updated transition model can achieve a lower error rate than the pretrained model. Therefore, the difference between the value function of the policy π under the transition function $\hat{T}_{on}$ and the value function under the true transition function T, is smaller than the difference between the value function of the policy under the transition function $\hat{T}_{off}$ and the value function under the transition function T.

$$\begin{aligned}&|\mathbb{E}_{s'\sim\hat{T}_{off}(s,a)}[V^{\pi}_{M}(s')] - \mathbb{E}_{s'\sim T(s,a)}[V^{\pi}_{M}(s')]|\\ &\geq |\mathbb{E}_{s'\sim\hat{T}_{on}(s,a)}[V^{\pi}_{M}(s')] - \mathbb{E}_{s'\sim T(s,a)}[V^{\pi}_{M}(s')]|,\end{aligned} \tag{10}$$

which concludes that $|G^{\pi}_{\hat{M}_{off}}(s,a)| \geq |G^{\pi}_{\hat{M}_{on}}(s,a)|$.

On the other hand, as the fine-tuned policy π_{on} is trained with the updated transition model $\hat{T}_{on}$, the residual item $|G^{\pi_{on}}_{\hat{M}_{on}}(s,a)|$ will be smaller than that of the offline policy. Therefore, for each pair (s,a), there is $|G^{\pi_{on}}_{\hat{M}_{on}}(s,a)| \leq |G^{\pi_{off}}_{\hat{M}_{on}}(s,a)| \leq |G^{\pi_{off}}_{\hat{M}_{off}}(s,a)|$.

As our policy collects samples with higher reward, it is trival that $\bar{\mathbb{E}}_{(s,a)\sim\rho^{\pi_{on}}_{\hat{T}_{on}}}[r(s,a)] \geq \bar{\mathbb{E}}_{(s,a)\sim\rho^{\pi_{off}}_{\hat{T}_{off}}}[r(s,a)]$. Summarizing the above inequalities, we can have the inequality of Equation 9.

REFERENCES

[1] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," 2020.

[2] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 6023–6029.

[3] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2021.

[4] S. Liu, K. C. See, K. Y. Ngiam, L. A. Celi, X. Sun, and M. Feng, "Reinforcement learning for clinical decision support in critical care: comprehensive review," *Journal of medical Internet research*, vol. 22, no. 7, p. e18477, 2020.

[5] S. Fujimoto, D. Meger, and D. Precup, "Off-policy deep reinforcement learning without exploration," in *International conference on machine learning*. PMLR, 2019, pp. 2052–2062.

[6] K. Zhao, Y. Ma, J. Liu, H. Jianye, Y. Zheng, and Z. Meng, "Improving offline-to-online reinforcement learning with q-ensembles," in *ICML Workshop on New Frontiers in Learning, Control, and Dynamical Systems*, 2023.

[7] S. Lee, Y. Seo, K. Lee, P. Abbeel, and J. Shin, "Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble," in *Conference on Robot Learning*. PMLR, 2022, pp. 1702–1712.

[8] A. Nair, A. Gupta, M. Dalal, and S. Levine, "Awac: Accelerating online reinforcement learning with offline datasets," 2020.

[9] H. Zhang, W. Xu, and H. Yu, "Policy expansion for bridging offline-to-online reinforcement learning," 2023.

[10] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Y. Zou, S. Levine, C. Finn, and T. Ma, "Mopo: Model-based offline policy optimization," *Advances in Neural Information Processing Systems*, vol. 33, pp. 14 129–14 142, 2020.

[11] M. Rigter, B. Lacerda, and N. Hawes, "Rambo-rl: Robust adversarial model-based offline reinforcement learning," *Advances in neural information processing systems*, vol. 35, pp. 16 082–16 097, 2022.

[12] T. Yu, A. Kumar, R. Rafailov, A. Rajeswaran, S. Levine, and C. Finn, "Combo: Conservative offline model-based policy optimization," *Advances in neural information processing systems*, vol. 34, pp. 28 954–28 967, 2021.

[13] J. Yang, X. Chen, S. Wang, and B. Zhang, "Model-based offline policy optimization with adversarial network," 2023. [Online]. Available: https://arxiv.org/abs/2309.02157

[14] J. Sun, Y. Jiang, J. Qiu, P. Nobel, M. J. Kochenderfer, and M. Schwager, "Conformal prediction for uncertainty-aware planning with diffusion dynamics model," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[15] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4rl: Datasets for deep data-driven reinforcement learning," 2020.

[16] Y. Wu, G. Tucker, and O. Nachum, "Behavior regularized offline reinforcement learning," 2019.

[17] H. Xu, L. Jiang, L. Jianxiong, and X. Zhan, "A policy-guided imitation approach for offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 4085–4098, 2022.

[18] A. Kumar, J. Fu, M. Soh, G. Tucker, and S. Levine, "Stabilizing off-policy q-learning via bootstrapping error reduction," *Advances in neural information processing systems*, vol. 32, 2019.

[19] J. Lyu, X. Ma, X. Li, and Z. Lu, "Mildly conservative q-learning for offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 1711–1724, 2022.

[20] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit q-learning," 2021.

[21] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1179–1191, 2020.

[22] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to trust your model: Model-based policy optimization," *Advances in neural information processing systems*, vol. 32, 2019.

[23] J. Wang, W. Li, H. Jiang, G. Zhu, S. Li, and C. Zhang, "Offline reinforcement learning with reverse model-based imagination," *Advances in Neural Information Processing Systems*, vol. 34, pp. 29 420–29 432, 2021.

[24] C. Lu, P. Ball, Y. W. Teh, and J. Parker-Holder, "Synthetic experience replay," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[25] Y. Liu, A. Swaminathan, A. Agarwal, and E. Brunskill, "Provably good batch off-policy reinforcement learning without great exploration," *Advances in neural information processing systems*, vol. 33, pp. 1264–1274, 2020.

[26] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "Morel: Model-based offline reinforcement learning," *Advances in neural information processing systems*, vol. 33, pp. 21 810–21 823, 2020.

[27] G. Zhou, S. Swaminathan, R. V. Raju, J. S. Guntupalli, W. Lehrach, J. Ortiz, A. Dedieu, M. Lázaro-Gredilla, and K. Murphy, "Diffusion model predictive control," 2024.

[28] M. Nakamoto, S. Zhai, A. Singh, M. Sobol Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, "Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[29] R. Willett, R. Nowak, and R. Castro, "Faster rates in regression via active learning," *Advances in Neural Information Processing Systems*, vol. 18, 2005.

[30] A. Korostelev, "On minimax rates of convergence in image models under sequential design," *Statistics & Probability Letters*, vol. 43, no. 4, pp. 369–375, 1999.

[31] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.

[32] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.

[33] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," *Advances in neural information processing systems*, vol. 34, pp. 20 132–20 145, 2021.

[34] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne." *Journal of machine learning research*, vol. 9, no. 11, 2008.